Java Inheritance & Interface Complete Notes

1. Inheritance Meaning

Inheritance in Java is a mechanism where one class acquires the properties and methods of another class.

2. Why Use Inheritance?

Code Reusability: reuse code.
Method Overriding: child changes parent method.
Polymorphism: one reference many forms.

3. Generalization, Specialization & Subtyping

Generalization combines common features into parent class.
Specialization creates child-specific features.
Subtyping means child becomes a type of parent.

Exam Definition: Inheritance is the process by which one class acquires the properties and behaviors of another class.

4. Parent & Child Class Terms

Term	Meaning
Parent / Super / Base Class	Class being inherited
Child / Sub / Derived Class	Class inheriting

5. extends and implements

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
}

interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

6. Access Control and Inheritance

Modifier	Same Class	Derived Class	Outside Class
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

7. Types of Inheritance

Type	Example
Single	A -> B
Multilevel	A -> B -> C
Hierarchical	A -> B and A -> C
Multiple	A + B -> C
Hybrid	Combination

8. Why Java Cannot Support Multiple Inheritance

Java avoids the Diamond Problem. If two parent classes have same methods, Java becomes confused which one to use. Therefore Java uses interfaces for multiple inheritance.

```
// INVALID JAVA CODE

class A {}
class B {}

class C extends A, B {
}
```

9. super Keyword

super refers to parent class object.

```
class Animal {
    Animal() {
        System.out.println("Animal Constructor");
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog Constructor");
    }
}
```

10. Internal Working of Inheritance

- Parent memory is created first.
- Child memory is created after.

- Child contains parent properties.
- Constructor chaining happens automatically.
- Overriding uses runtime polymorphism.

Important: Parent classes stay at the top because child classes depend on them.

11. Advantages & Disadvantages

Advantages:

- Code reuse
- Easy maintenance
- Less duplicate code
- Supports polymorphism

Disadvantages:

- Complex hierarchy
- Tight coupling
- Hard debugging

12. Interface

An interface is a blueprint of a class that contains abstract methods. Interfaces are used to achieve abstraction and multiple inheritance.

Exam Definition: An interface is a collection of abstract methods used to achieve abstraction and multiple inheritance.

13. Interface Example

```
interface Animal {
    void sound();
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

14. Multiple & Hybrid Using Interface

```
interface A {
    void showA();
}

interface B {
    void showB();
}

class C implements A, B {
```

```
    public void showA() {}

    public void showB() {}
}
```

15. Real World Uses

- Banking systems
- Android apps
- Game engines
- Enterprise software

16. Last Minute Exam Notes

• Inheritance uses extends.

• Interface uses implements.

• Java does not support multiple inheritance with classes.

• super accesses parent class.

• Overriding enables runtime polymorphism.